

TECHNICAL REFERENCE GUIDE

Qwik Guide: Unsupervised Learning

Clustering · K-means · Hierarchical · DBSCAN · PCA · Anomaly Detection · Association Rules

A beginner-friendly companion for **ML for Beginners**.

Plain-language definitions, worked processes, and diagrams — a partner volume to the Supervised Learning guide.

Unsupervised Learning, From the Ground Up

Supervised learning hands the algorithm **labeled examples** — inputs paired with the correct answers. **Unsupervised learning** works with **unlabeled examples**: there is **no prior knowledge** of the right answer, no target column to predict. The algorithm's job is to **find structure that is already in the data** — natural groups, simple summaries, unusual points, or items that tend to appear together.

This guide keeps the language plain, defines every term as it appears, walks through each process step by step, and adds a picture wherever one helps. It is the companion to the Supervised Learning guide, so we compare the two throughout.

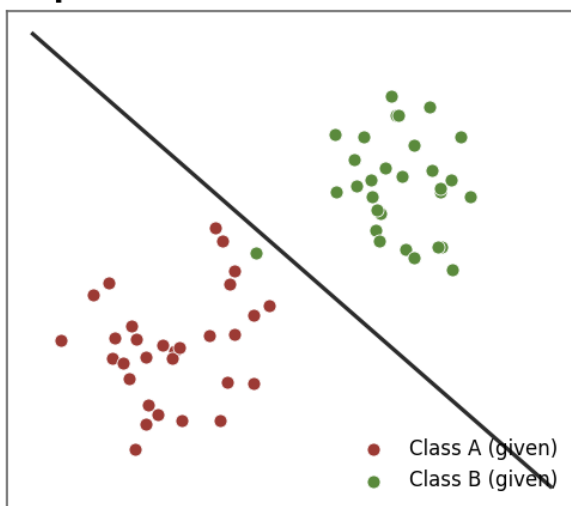
TIP Read this after the classification-vs-regression section of the Supervised guide. The single biggest idea here: with **no labels**, there is no "correct" answer to score against — so we look for patterns instead of predicting a known target.

1 Supervised vs. Unsupervised

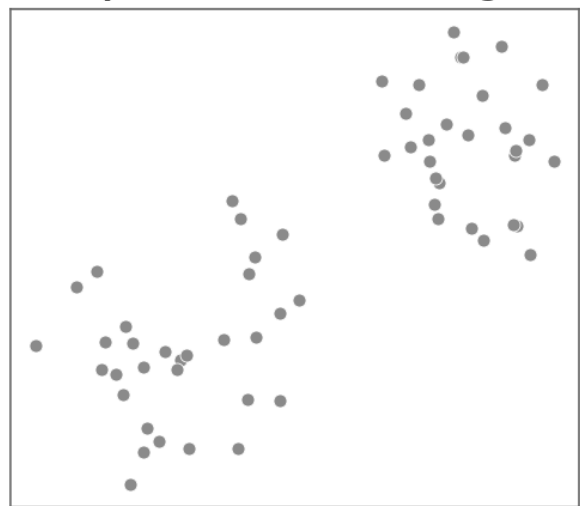
In **supervised** learning you already know the label for each training example, and the model learns a rule that maps new inputs to those labels. In **unsupervised** learning the labels are missing — the model groups, compresses, or ranks the data using only the input features themselves.

Labels given vs. no labels given

Supervised → learns from labels



Unsupervised → no labels given



Left: supervised learning is told which class each point belongs to. Right: unsupervised learning sees the same points with the colors (labels) stripped away, and must discover the grouping on its own.

Key vocabulary

- **Unlabeled examples** — data with **no prior knowledge** of the answer attached; the model finds patterns instead of matching known labels.
- **Feature** — one measured attribute of an observation (e.g. height, price, word count).
- **Observation** — one data point / row, described by its features.
- **Common uses** — customer segmentation, anomaly detection, compressing data for visualization, and market-basket analysis.

2 Preparing the Data

Most clustering algorithms measure **distance** between points, so preparation matters more than in many supervised tasks. If one feature runs 0–1 and another runs 0–100,000, the big one dominates every distance unless you rescale.

What to handle first

- **Missing values** — fill or drop them before measuring distances.
- **Numerical vs. categorical features** — categories usually need encoding into numbers.
- **Standardization** — put every feature on the same scale (e.g. scikit-learn's `StandardScaler`) so no single feature dominates.
- **Euclidean distance** — the straight-line distance between two points; the default "how far apart" measure for k-means.
- **Exploratory plots** — look at the data before clustering; a scatter plot often reveals the rough shape.

TIP Scaling is the step beginners skip most often. Because distance drives clustering, an unscaled feature can silently decide every cluster. Scale first, cluster second.

3 Clustering Fundamentals

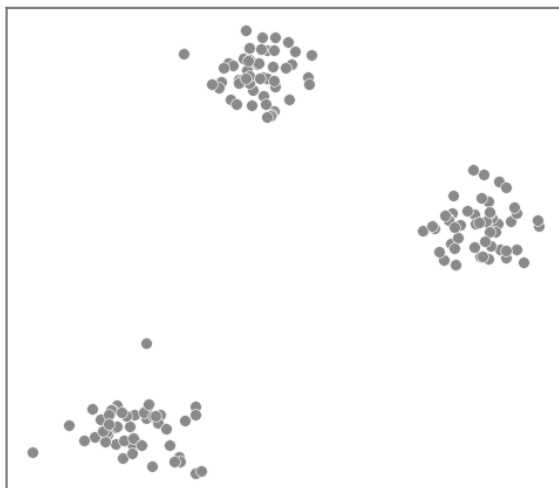
Clustering groups similar observations together **without predefined class labels**. Points inside a cluster should be close together; different clusters should sit well apart.

Centroid

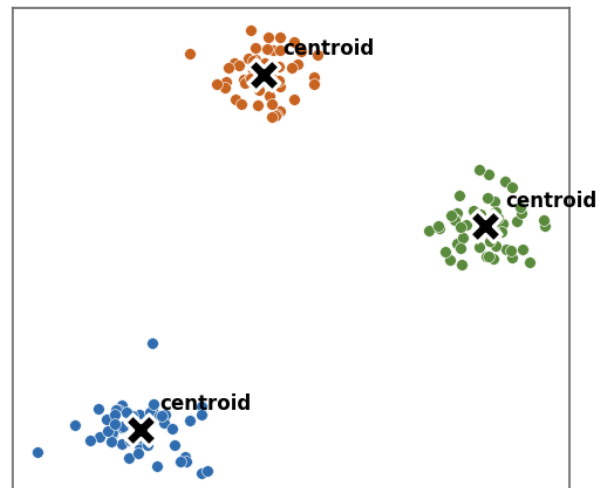
A **centroid** is the **point at the center of a cluster** — the average position of all its members. Many algorithms represent a whole cluster by its centroid.

From unlabeled points to clusters with centroids

Before: unlabeled points



After: k-means clusters + centroids

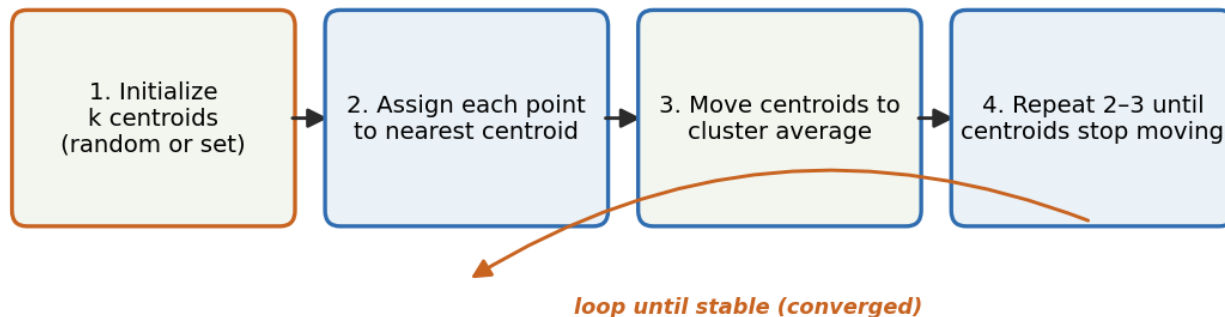


The centroid (X) sits at the middle of each cluster — the average of its members' positions.

4 K-means Clustering

K-means is the best first clustering algorithm to learn. You pick **k**, the number of clusters, and the algorithm finds **k** centroids and assigns every point to its nearest one.

The k-means algorithm, step by step



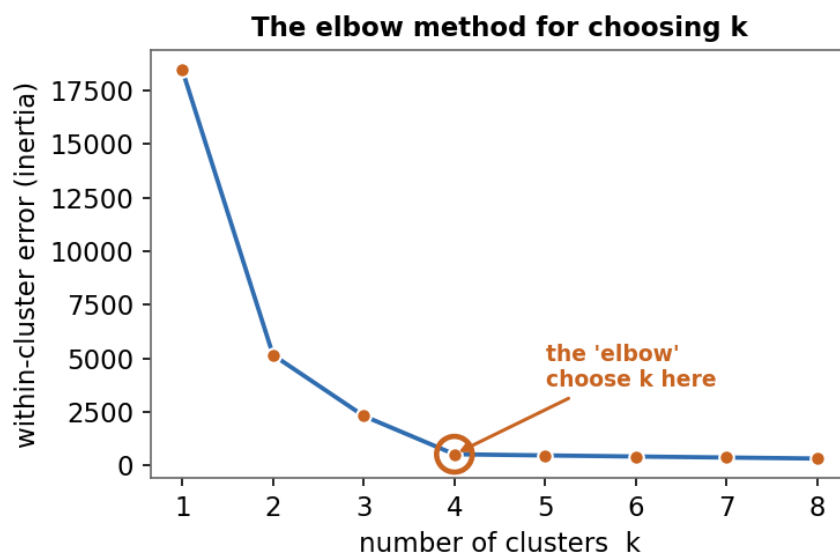
The process is a simple repeating loop:

- 1 **Initialize** **k** centroids — placed either at **random positions** or at **user-defined** ones.
- 2 **Classify each example** using the centroids by finding the **closest** cluster (nearest centroid).
- 3 **Move all centroids** by taking the **average** of each cluster's current members.
- 4 **Repeat** steps 2–3 until the centroids stop moving (the algorithm has **converged**).

TIP Watch the order: memberships are updated **from** the centroids (step 2), then centroids are recomputed **from** the memberships (step 3). Both update every full pass — not "half the memberships" at a time.

Choosing k — the elbow method

K-means needs **k** up front. Plot the within-cluster error against several values of **k**; it falls sharply, then flattens. The bend — the "elbow" — is a good choice for **k**.



Error drops fast, then levels off. The elbow marks the point where adding more clusters stops helping much.

Why k-means is popular

- **Fast and simple** to run and understand.
- Good for **rapid prototyping** — a quick first look at structure.
- Useful as **preprocessing** for a more complex clustering algorithm.

Where k-means struggles

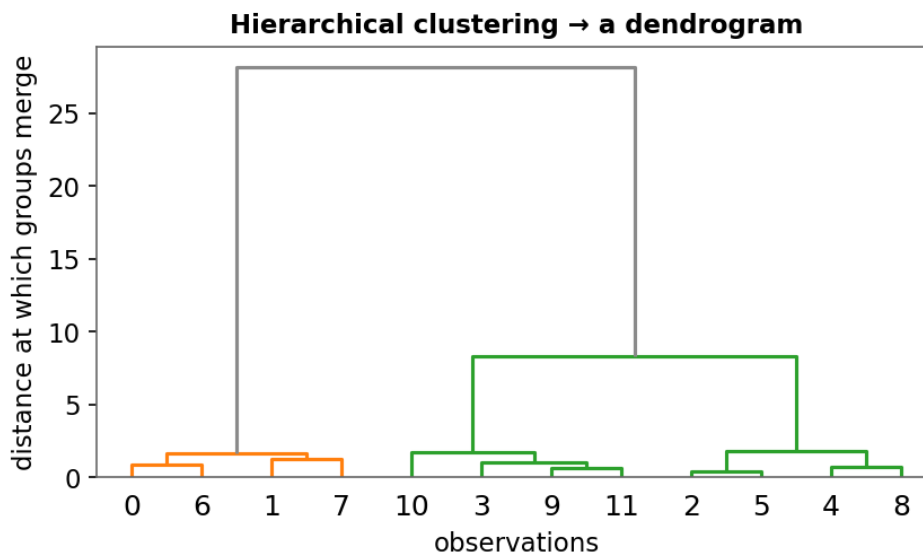
- You must choose **k** in advance.
- It assumes roughly round, similarly sized clusters — **irregular shapes and outliers** throw it off.
- Results depend on the **random initialization**; different starts can give different clusters, so it is usually run several times and does **not** try every possible initial placement systematically.

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

X = StandardScaler().fit_transform(X) # scale first — distance-based
model = KMeans(n_clusters=3, n_init=10, random_state=0)
labels = model.fit_predict(X) # cluster id for each point
centroids = model.cluster_centers_ # the k centroids
```

5 Hierarchical Clustering

Hierarchical clustering builds a **tree of nested groups** instead of a single flat set. The agglomerative version starts with every point as its own cluster, then repeatedly merges the two closest clusters until one remains. The result is drawn as a **dendrogram**.



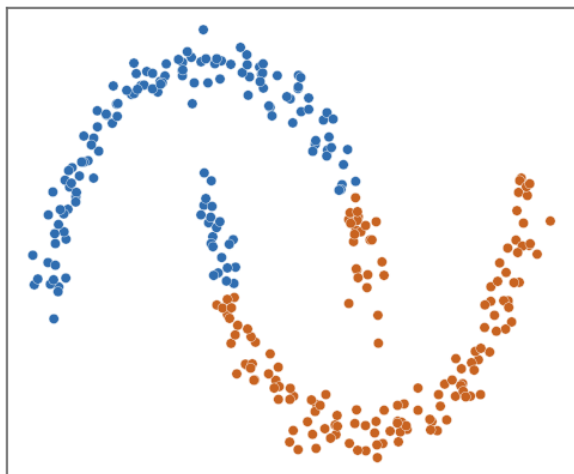
A dendrogram: the height of each join shows how far apart two groups were when merged. "Cut" it at a chosen height to read off that many clusters — no need to fix k in advance.

Linkage is the rule for the distance between two groups — single, complete, average, or Ward. Ward (minimizing added variance) is a solid default.

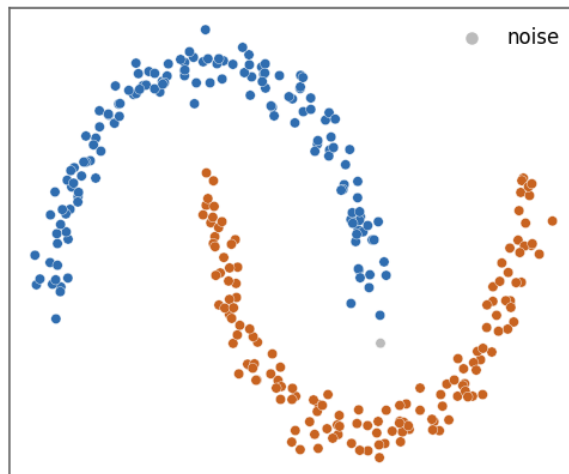
6 Density-Based Clustering: DBSCAN

DBSCAN groups points that are **packed closely together** and labels lonely points as **noise**. Two things make it stand out: it finds **irregular shapes**, and it does **not** need the number of clusters up front.

k-means: struggles with shapes



DBSCAN: finds shapes + noise



On crescent-shaped data k-means splits the shapes wrongly, while DBSCAN follows the density and flags stray points as noise (grey).

Two parameters control it: **eps** (how close counts as "neighboring") and **min_samples** (how many neighbors make a dense region). Points in no dense region are returned with the noise label `-1`.

7 Evaluating Clustering Results

Because unsupervised data usually has **no correct answers** to check against, evaluation takes extra care. A common numeric guide is the **silhouette score**, which rewards clusters that are **compact** and **well separated**.

What to examine

- **Silhouette score** — compares each point's fit to its own cluster vs. the nearest other cluster.
- **Cluster size, separation, and compactness.**
- **Stability** — do the clusters hold when parameters change slightly?
- **Real-world meaning** — do the clusters correspond to something you care about?

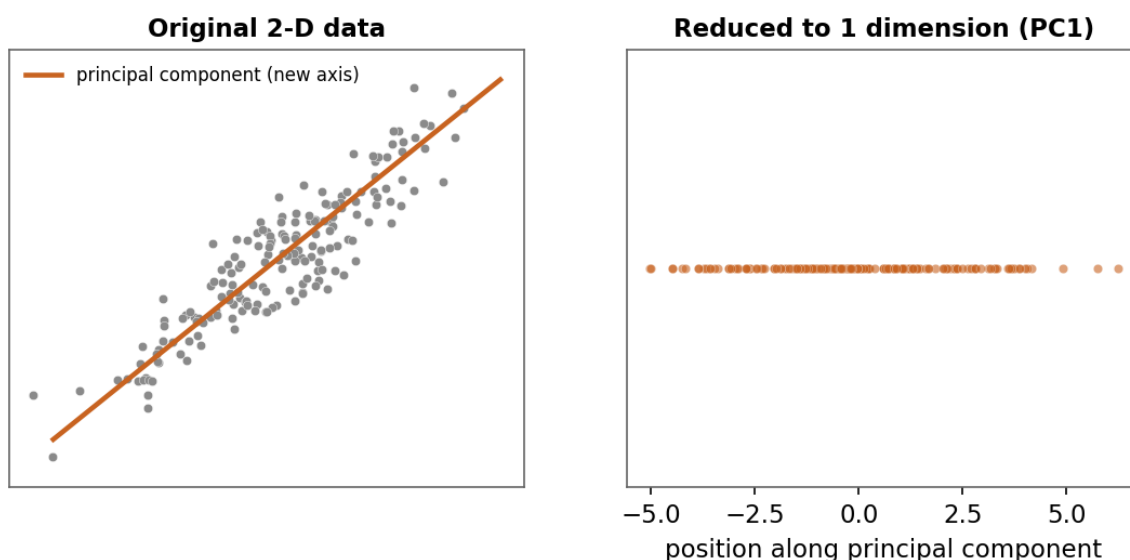
TIP A high silhouette score does **not** automatically mean the clusters are useful. Always ask whether they answer your actual question.

8 Dimensionality Reduction & PCA

Dimensionality reduction creates a smaller representation of the data by cutting the number of features while keeping as much of the useful signal as possible. It chiefly reduces **collinearity** — the redundancy between features that measure nearly the same thing — and makes data easier to plot and to model.

Principal Component Analysis (PCA)

PCA is the best introductory method. It achieves its result through **linearity**: it **maps the data onto a different axis** — new axes called **principal components** — chosen so the first captures the most variance, the second the next most, and so on.



PCA rotates to a new axis (the principal component) that follows the data's main spread, then projects onto it — turning 2-D data into 1-D while keeping most of the variation.

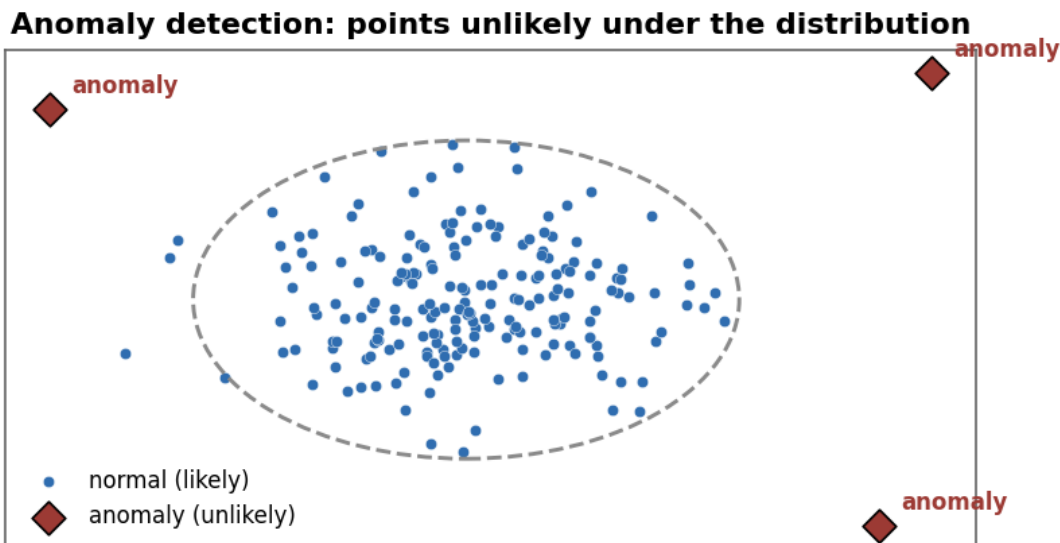
Key ideas

- **Features vs. principal components** — components are new combinations of the original features, not the originals themselves.
- **Explained variance** — how much of the data's spread each component keeps.
- **Eliminating dimensions** — the variance carried by the dropped dimensions is effectively **deleted**; you trade a little information for a much smaller, cleaner dataset.
- **Scale first** — PCA usually follows feature scaling, and the resulting components can be hard to interpret.

```
from sklearn.decomposition import PCA
X2 = PCA(n_components=2).fit_transform(X)  # reduce to 2-D for a plot
```


9 Anomaly & Outlier Detection

Anomaly detection finds observations that differ from most of the data. The core idea is probability: **if the occurrence of a point is unlikely according to a distribution, it can be identified as an anomaly.**

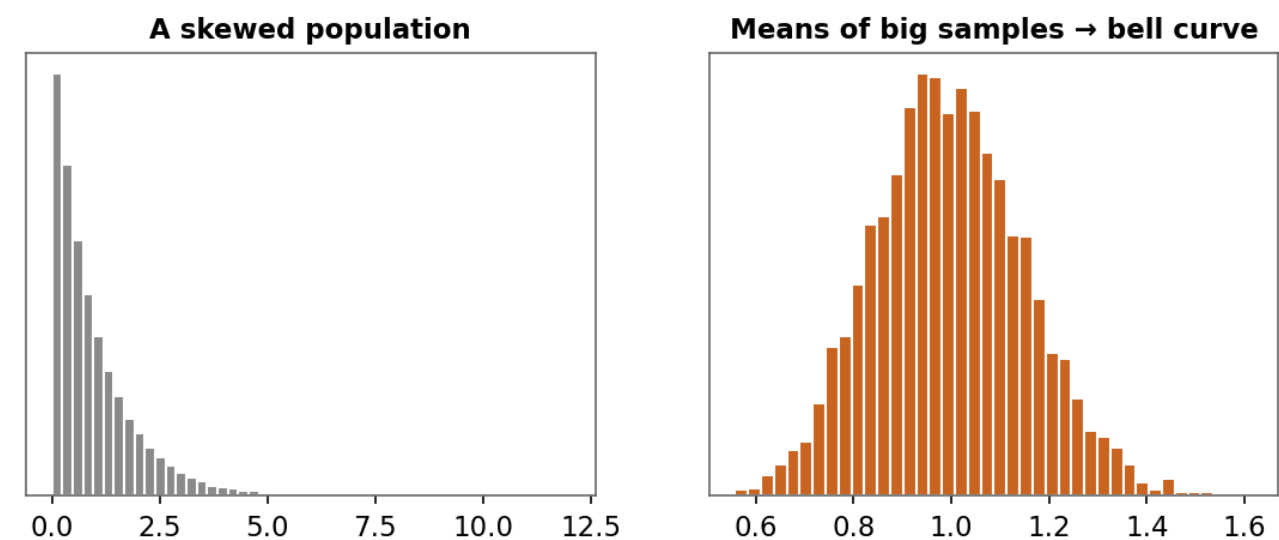


Most points fall in the dense region the model considers "likely." Points far out in the low-probability tail are flagged as anomalies.

Beginner-friendly methods: **Local Outlier Factor** and **Isolation Forest** before the heavier One-Class SVM. Uses include fraud and unusual-transaction detection.

10 A Statistical Aside: the Central Limit Theorem

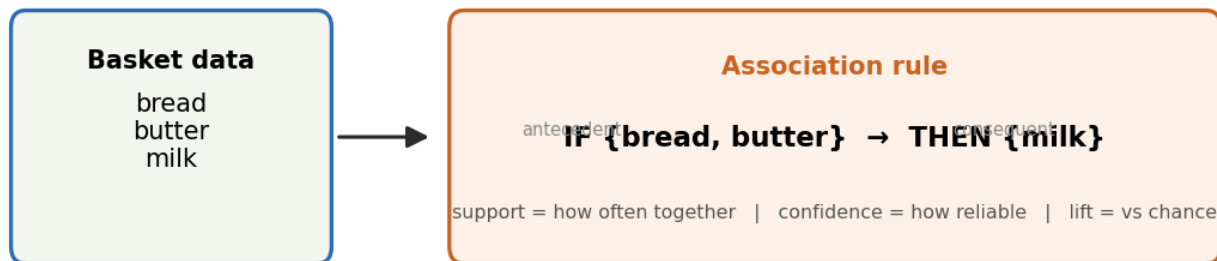
The **central limit theorem** is useful because **big samples of random, independent variables tend to be distributed normally** (the familiar bell curve) — even when the underlying population is skewed. That is why the normal distribution shows up so often, and why many "likely vs. unlikely" judgments in anomaly detection lean on it.



Technical Reference Series
Left: a lopsided population. Right: take many large samples and average each — the averages pile up into a bell curve.

11 Association-Rule Learning

Association rules are **correlations and relationships between variables in a dataset** — they discover items or events that frequently occur together, such as products bought in the same transaction ("market-basket analysis").



A rule has an **antecedent** (the "if" items) and a **consequent** (the "then" item), scored by support, confidence, and lift.

Prior knowledge (the a-priori quantities)

To mine rules with the Apriori method you work from a few required **prior-knowledge** quantities. The three core ones are:

- **Support** — how often an itemset appears in all transactions (its frequency).
- **Confidence** — how reliable the rule is: given the antecedent, how often the consequent follows.
- **Itemset size** — how many items are grouped together in the set being scored.

Together these define a **frequent itemset**. A **feature**, by contrast, is a general attribute of an observation — it is **not** one of the required a-priori components for rule mining.

TIP Lift compares the rule to pure chance (lift > 1 means the items appear together more than expected). And remember: a strong association shows **correlation, not causation**.

```
from mlxtend.frequent_patterns import apriori, association_rules
freq = apriori(df, min_support=0.02, use_colnames=True)
rules = association_rules(freq, metric="confidence", min_threshold=0.5)
```

12 Quick Reference

Concept	Type	Remember this
Centroid	Clustering	The point at the center of a cluster (its average)
Unlabeled examples	Foundations	No prior knowledge of the answer; find patterns, don't predict labels
K-means	Clustering	Loop: assign to nearest centroid → move centroids to the average → repeat
Elbow method	Clustering	Pick k at the bend where error stops dropping fast
Hierarchical	Clustering	Merge nearest groups into a tree; read clusters off the dendrogram
DBSCAN	Clustering	Density-based; finds odd shapes and labels noise; no k needed
Silhouette score	Evaluation	Rewards compact, well-separated clusters (high ≠ always useful)
PCA	Dim. reduction	Linear map to a new axis ; reduces collinearity; dropped-dimension variance is deleted
Anomaly detection	Detection	Unlikely under the distribution → flagged as an anomaly
Central limit theorem	Statistics	Big samples of independent variables → normal distribution
Association rules	Pattern mining	Relationships between variables; scored by support, confidence, lift
A-priori components	Pattern mining	Itemset size, support, confidence (a feature is not one of these)

Unsupervised vs. supervised — at a glance

	Supervised	Unsupervised
Data	Labeled examples	Unlabeled examples (no prior knowledge)
Goal	Predict a known target	Find hidden structure
Typical tasks	Classification, regression	Clustering, dimensionality reduction, anomaly detection, association rules
Scoring	Compare to correct answers	Internal measures (e.g. silhouette) + human judgment

★ Further Resources

Arranged in a recommended learning order — foundations first, then data prep, clustering, and the individual algorithms. Continued on the next page.

Foundations: what is unsupervised learning?

Google: What Is Machine Learning? — <https://developers.google.com/machine-learning/intro-to-ml/what-is-ml>

IBM: What Is Unsupervised Learning? — <https://www.ibm.com/think/topics/unsupervised-learning>

Google Machine Learning Courses — <https://developers.google.com/machine-learning/>

Preparing data

pandas: Getting Started Tutorials — https://pandas.pydata.org/docs/getting_started/intro_tutorials/

scikit-learn: Preprocessing Data — <https://scikit-learn.org/stable/modules/preprocessing.html>

scikit-learn: Importance of Feature Scaling — https://scikit-learn.org/stable/auto_examples/preprocessing/plot_scaling_importance.html

Clustering fundamentals

Google: Introduction to Clustering — <https://developers.google.com/machine-learning/clustering>

Google: Types of Clustering Algorithms — <https://developers.google.com/machine-learning/clustering/clustering-algorithms>

scikit-learn: Clustering User Guide — <https://scikit-learn.org/stable/modules/clustering.html>

StatQuest Video Index — https://statquest.org/video_index.html

K-means clustering

Google: What Is K-means Clustering? — <https://developers.google.com/machine-learning/clustering/kmeans/overview>

scikit-learn: KMeans Documentation — <https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html>

scikit-learn: K-means on Handwritten Digits — https://scikit-learn.org/stable/auto_examples/cluster/plot_kmeans_digits.html

Hierarchical clustering

scikit-learn: Hierarchical Clustering — <https://scikit-learn.org/stable/modules/clustering.html>

Comparing Hierarchical Linkage Methods — https://scikit-learn.org/stable/auto_examples/cluster/plot_linkage_comparison.html

SciPy: Dendrogram Documentation — <https://docs.scipy.org/doc/scipy/reference/generated/scipy.cluster.hierarchy.dendrogram.html>

Density-based clustering: DBSCAN

scikit-learn: DBSCAN Demonstration — https://scikit-learn.org/stable/auto_examples/cluster/plot_dbscan.html

scikit-learn: DBSCAN Documentation — <https://scikit-learn.org/stable/modules/generated/sklearn.cluster.DBSCAN.html>

Clustering Algorithm Comparison — https://scikit-learn.org/stable/auto_examples/cluster/plot_cluster_comparison.html

Evaluating clustering results

Google: Clustering Course — <https://developers.google.com/machine-learning/clustering>

scikit-learn: Silhouette Analysis — https://scikit-learn.org/stable/auto_examples/cluster/plot_kmeans_silhouette_analysis.html

scikit-learn: Silhouette Score — https://scikit-learn.org/stable/modules/generated/sklearn.metrics.silhouette_score.html

Dimensionality reduction and PCA

scikit-learn: Unsupervised Dimensionality Reduction — https://scikit-learn.org/stable/modules/unsupervised_reduction.html

PCA on the Iris Dataset — https://scikit-learn.org/stable/auto_examples/decomposition/plot_pca_iris.html

scikit-learn: PCA Documentation — <https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html>

StatQuest Video Index — https://statquest.org/video_index.html

Anomaly and outlier detection

scikit-learn: Novelty and Outlier Detection — https://scikit-learn.org/stable/modules/outlier_detection.html

Local Outlier Factor Example — https://scikit-learn.org/stable/auto_examples/neighbors/plot_lof_novelty_detection.html

Isolation Forest Example — https://scikit-learn.org/stable/auto_examples/ensemble/plot_isolation_forest.html

Association-rule learning

IBM: Unsupervised Learning and Association Rules — <https://www.ibm.com/think/topics/unsupervised-learning>

mlxtend: Apriori Algorithm — https://rasbt.github.io/mlxtend/user_guide/frequent_patterns/apriori/

mlxtend: Association Rules — https://rasbt.github.io/mlxtend/user_guide/frequent_patterns/association_rules/

Practice datasets and environments

Google Colab — <https://colab.research.google.com/>

UCI Machine Learning Repository — <https://archive.ics.uci.edu/datasets>

scikit-learn Dataset Loaders — <https://scikit-learn.org/stable/api/sklearn.datasets.html>

scikit-learn Clustering Examples — https://scikit-learn.org/stable/auto_examples/cluster/index.html

TIP Recommended six-week path. (1) ML fundamentals & data prep → (2) k-means → (3) hierarchical & DBSCAN → (4) cluster evaluation → (5) PCA & dimensionality reduction → (6) anomaly detection & association rules. Implement one small notebook after each area rather than reading everything first — try clustering Iris with k-means, reducing Wine with PCA, and finding product combinations with Apriori.

★ Two Starter Projects, Explained

The six-week path ends with two classic first exercises. Here is what each one actually means — one shows **dimensionality reduction** on real data, the other shows **association-rule learning** on basket data.

Reducing Wine with PCA

The **Wine dataset** is a standard practice dataset where each wine is described by **13 chemical features** (alcohol, acidity, color intensity, and so on). "Reducing with PCA" means using **Principal Component Analysis** to compress those 13 features down to just **2**, so you can plot every wine on a simple 2-D scatter chart and visually watch the wine types separate into groups — while keeping most of the meaningful variation.

Why it's a good first project It turns a table you can't picture (13 columns) into one clear chart, making PCA's payoff obvious: fewer dimensions, most of the signal kept.

```
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

X = load_wine().data           # 13 chemical features
X = StandardScaler().fit_transform(X) # scale first
X2 = PCA(n_components=2).fit_transform(X) # 13 -> 2 for a scatter plot
```

Finding product combinations with Apriori

The **Apriori algorithm** mines **association rules** from transaction data (like shopping baskets). "Finding product combinations" means discovering which items tend to be bought together — for example, *"people who buy bread and butter also tend to buy milk"* — by scoring itemsets with **support**, **confidence**, and **lift**.

Why it's a good first project It connects the association-rule vocabulary to a concrete, intuitive result: a ranked list of "if this, then that" product combinations you can read at a glance.

```
from mlxtend.frequent_patterns import apriori, association_rules

freq = apriori(baskets, min_support=0.02, use_colnames=True)
rules = association_rules(freq, metric="lift", min_threshold=1.0)
rules.sort_values("confidence", ascending=False) # best combos first
```